

임무컴퓨터

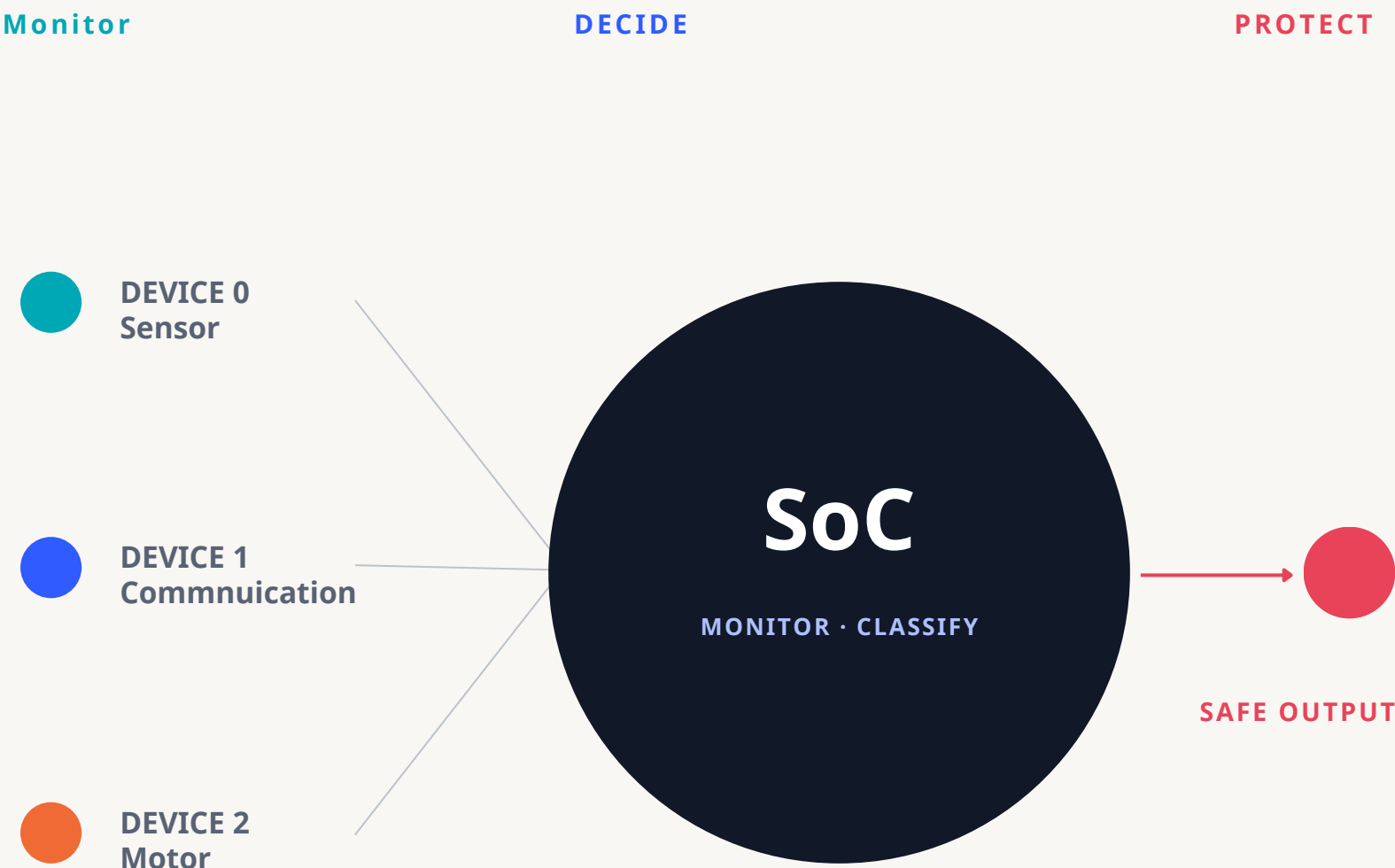
상태 감시, 고장 대응

SoC

다중 장치 Fault 감지 · 등급화 · 안전 제어
RTL과 MicroBlaze V, GUI를 연결한 Mission SoC

온디바이스 AI 반도체 설계 3기

박기태 김민석 이재운



3

monitored devices

4

safety states

Table of Contents

01.프로젝트 개요

- 프로젝트 배경
- 프로젝트 목표
- 역할 분담

02. 핵심 설계 정책

- 장치별 감시 기준
- Fault 우선순위
- Fault 지속 판정 구조
- Recovery 구조

03. 시스템 설계

- 전체 시스템 아키텍처
- Heartbeat Monitor
- Fault Manager
- Safety Controller

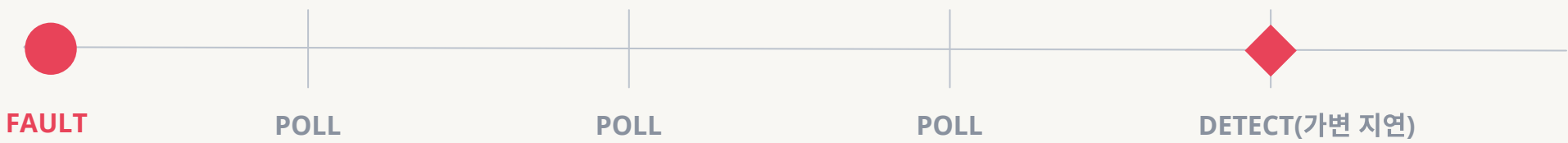
04. 동작 검증 및 결과

- GUI 화면 설명
- 시연 영상
- 검증 환경
- Troubleshooting
- 합성·Timing·자원 사용량

프로젝트 배경

Polling 주기와 CPU 부하에 따라 달라지는 안전 대응 시간

기존 방식



CPU POLLING 응답 시점이 폴링 주기와 부하에 따라 달라짐

Mission SoC



RTL = 안전경로 동기화 이후 FM 1clk + SC 1clk의 결정적 경로

안전 판단 하드웨어

- 설정된 Timeout에서 검출
- 중요도와 지속시간을 분리 평가
- CPU 개입 없이 출력 차단
- 복구는 별도 정책으로 제한

프로젝트 목표

Fault 감지부터 선택 차단, 상태별 자동 복귀와 SAFE_MODE 수동 해제까지 하나의 SoC로 구현

01 실시간 감시

- Device 0 · 1 · 2
- Heartbeat와 실제 Timeout 판정

02 Fault 등급화

- 중요도와 지속시간 기반
- Critical · Multi · Persistent

03 출력 보호

- 고장 범위만 선택 차단
- D0 → 110
- D1 → 101
- Level3 → 000

04 상태 복귀

- Warning · Degraded 자동 복귀
- Safe_Mode Level 0 + Manual Reset

역할 분담

3종 Custom IP의 책임 개발과 공통 인터페이스 기반의 시스템 통합·검증

A 김민석

HEARTBEAT MONITOR

- 비동기 Heartbeat 입력 동기화
- 장치별 Alive / Timeout 상태 판정
- AXI 레지스터 및 Testbench를 통한
감시 기능 검증

B 이재운

FAULT MANAGER

- 장치별 Fault 정보 통합
- 중요도 · 지속 횟수 기반 등급화
- Device ID, Fault Code, Fault Level
생성 및 AXI 전달

C 박기태

SAFETY CONTROLLER

- Fault 기반 4-State FSM 제어
- 고장 장치 선택 차단 및 SAFE_MODE 진입
- 정상 신호 지속 시 자동 복구, Manual
Reset 기반 해제

공통 개발 사항

- Vivado Block Design 기반 3종 Custom IP 연결
- UART 기반 상태 출력 및 PC GUI 제어
- 정상·Timeout·Error·Critical Fault 통합 시나리오 검증

장치별 감시 기준

장치 중요도에 따른 Heartbeat 주기·Timeout 기준과 Fault 처리 우선순위

	D0 일반 센서 처리	D1 통신 장치	D2 모터 *핵심 장치
HEARTBEAT	100ms	200ms	50ms
TimeOut	300ms	600ms	150ms
Fault 정책	지속 Fault 판정 Persist Limit 적용	지속 Fault 판정 Persist Limit 적용	지속 판정 없이 Level 3

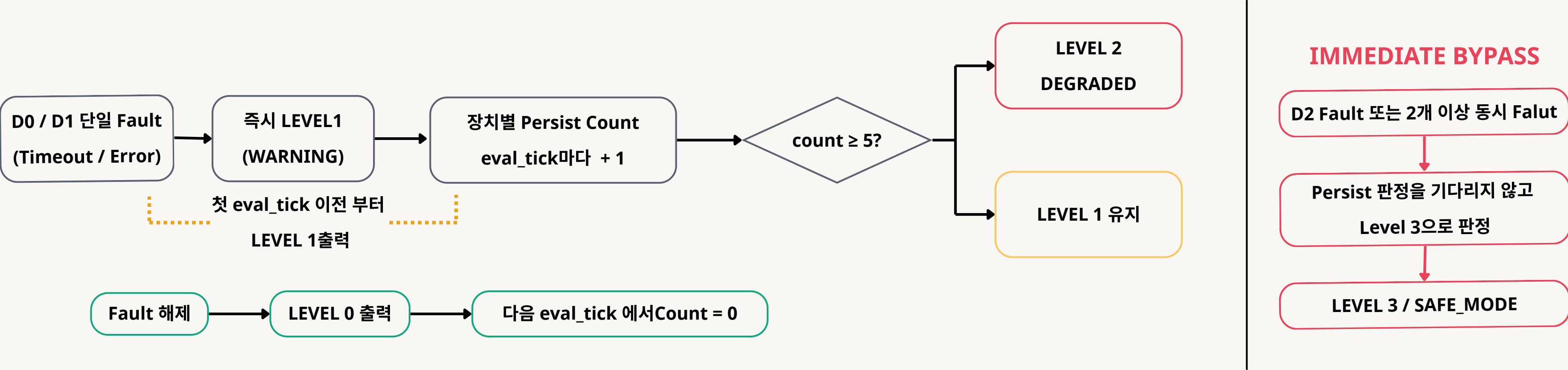
Fault 우선순위

Critical 조건부터 일반 Fault까지의 단계별 판정 순서

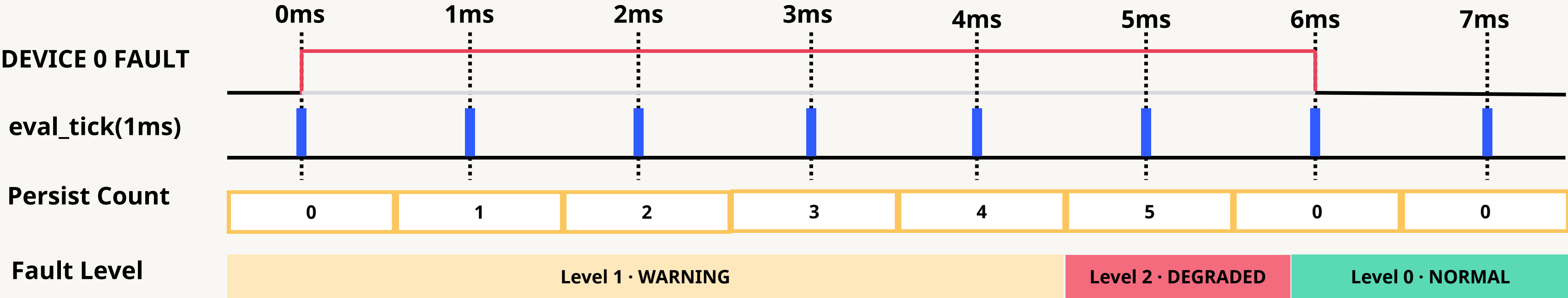
순서		판정 조건	Fault Manager 출력
1	CRITICAL Mask Hit	(device_fault & critical_mask) != 0 기본 Critical_Mask = 3'b100 → Device 2의 모든 Fault	Level 3 FAULT_CRITICAL
2	MULTI DEVICE	critical_condition = 0 · popcount(device_fault) ≥ 2	Level 3 FAULT_MULTI_DEVICE
3	PERSISTENT Fault	단일 비 Critical Fault · Persist Count ≥ Persist Limit	Level 2 원인 코드
4	TRANSIENT Fault	단일 비 Critical Fault · Persist Count < Persist Limit	Level 1 원인 코드
5	Normal	device_Fault = 0	Level 0 Fault_NONE

Fault 지속 판정 구조

일시적인 Fault와 지속 Fault를 구분하여 불필요한 상태 전환을 방지

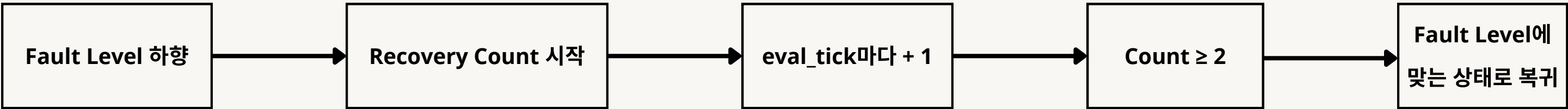


동작예시

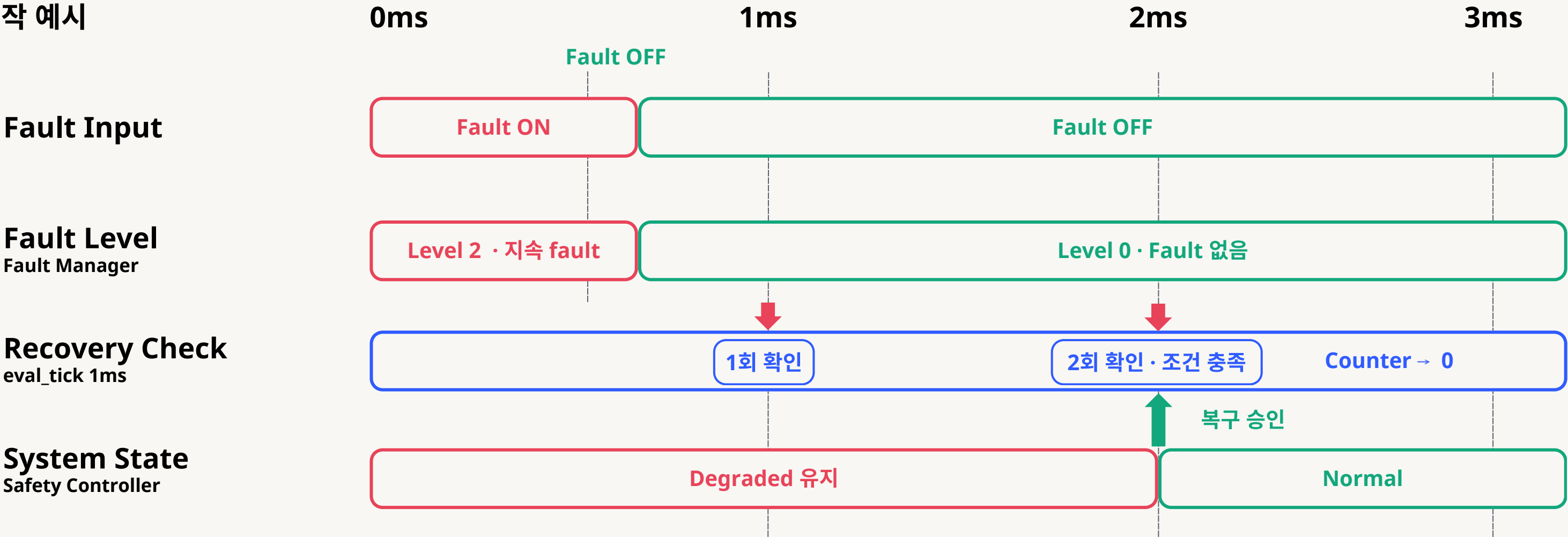


Recovery 구조

Fault Level이 낮아진 뒤, 완화된 Level을 연속 확인하고 상태 복귀를 허용하는 정책

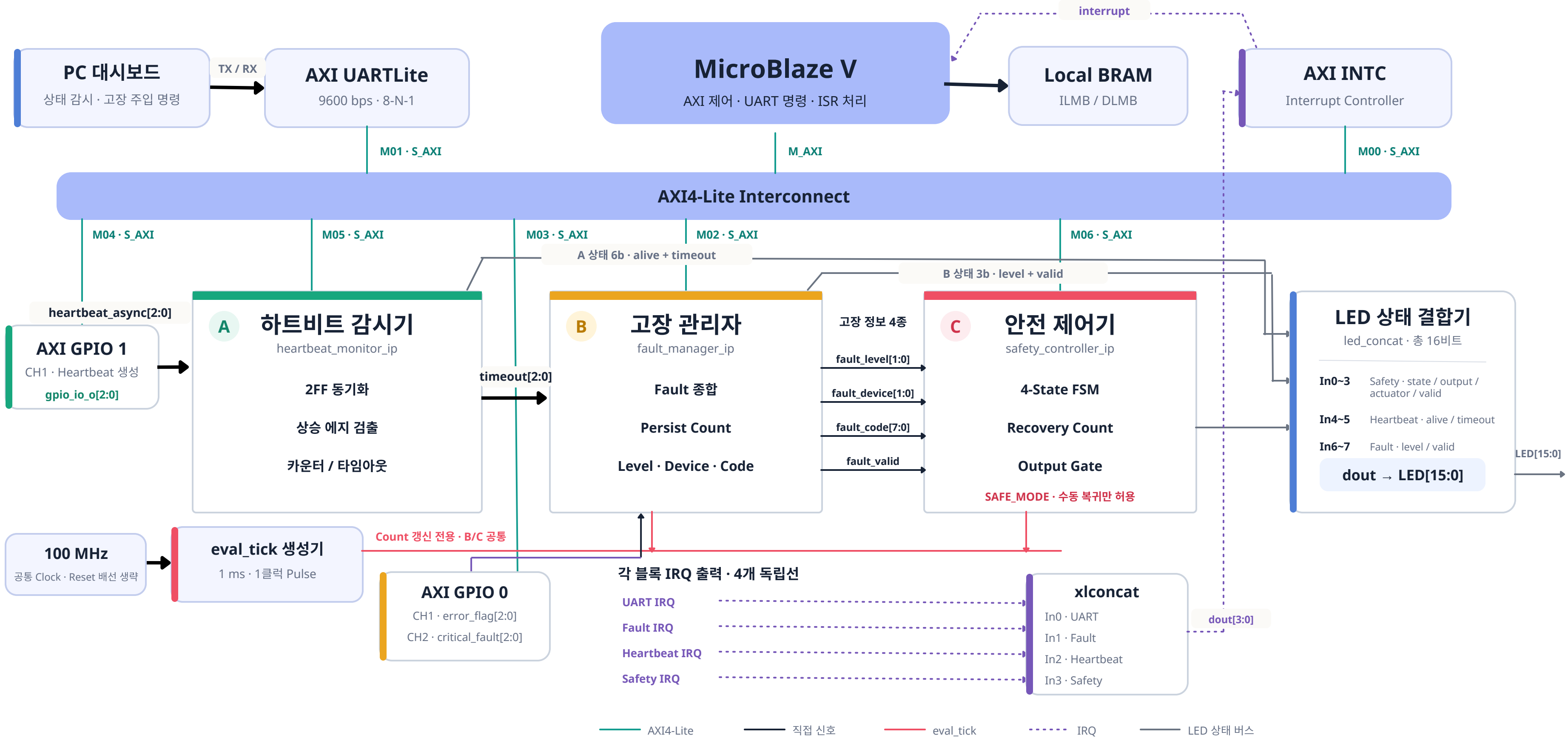


동작 예시



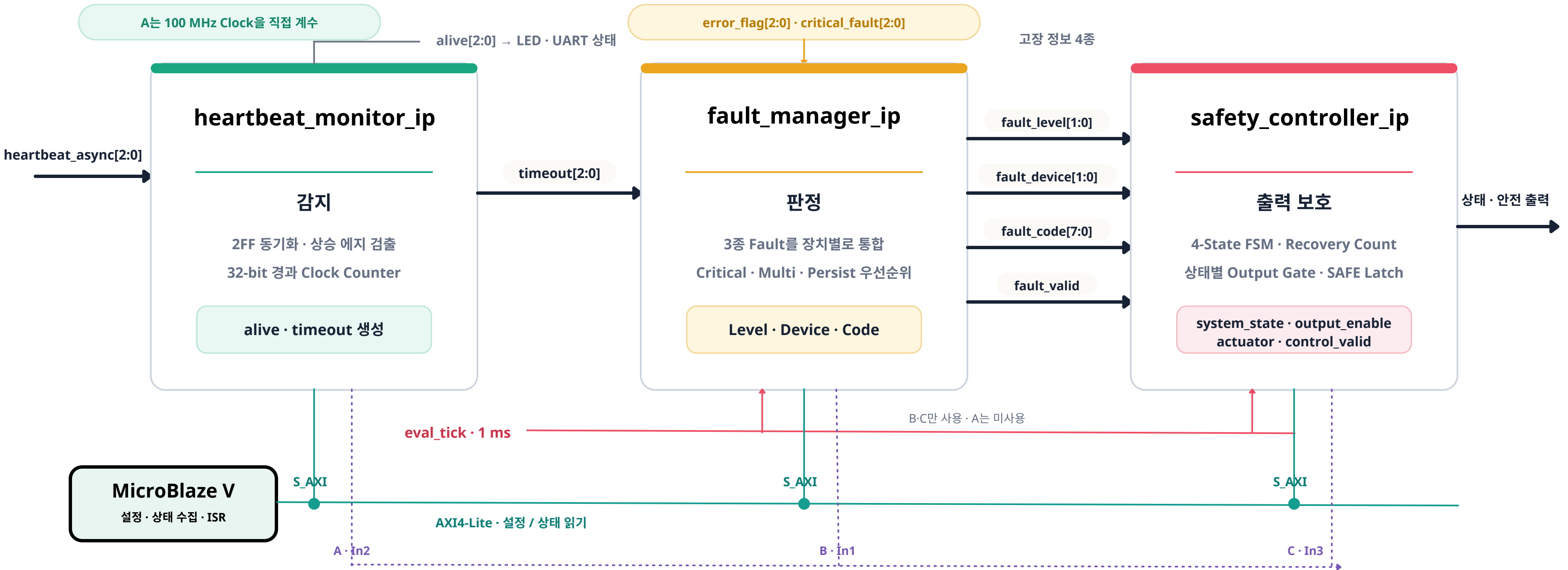
전체 시스템 아키텍처

MicroBlaze V를 중심으로 감시 · 고장 판정 · 안전 제어 · 상태 표시를 통합



커스텀아이피 연결 구조

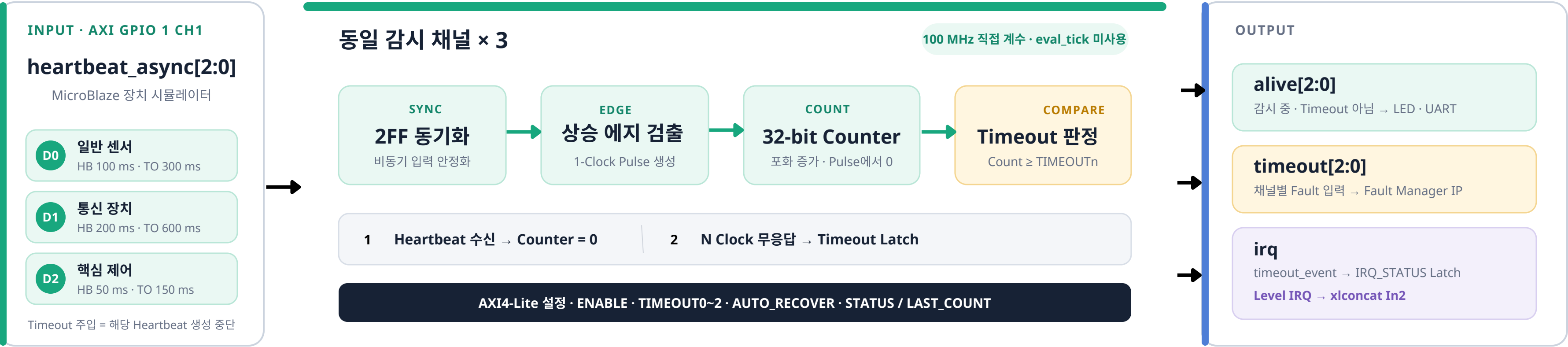
감지 → 판정 → 출력 보호로 이어지는 하드웨어 신호 계약



IRQ 집계
UART In0 · B In1 · A In2 · C In3
xlconcat → AXI INTC → MicroBlaze V

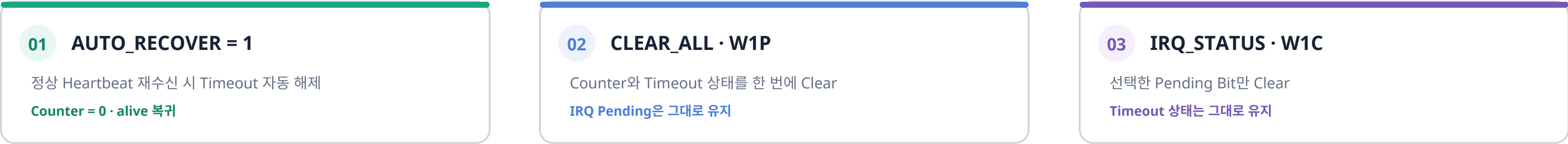
하트비트 감시 IP

3채널 비동기 Heartbeat를 안정화하고, 마지막 상승 에지 이후의 경과 Clock으로 Alive·Timeout을 판정



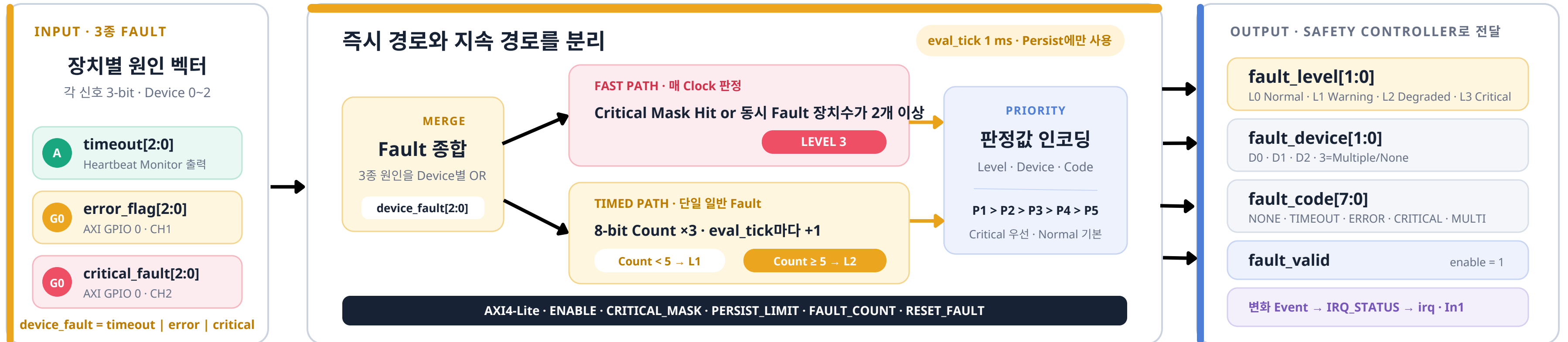
상태 해제와 IRQ 해제를 분리

재수신·명령·W1C의 역할을 명확히 분리해 오동작을 방지



고장 관리자 IP

Timeout·Error·Critical 입력을 장치별로 통합하고, 중요도와 지속 시간으로 Level·Device·Code를 결정



판정 정책과 상태 해제 경계

Mask·Reset·IRQ의 역할을 분리해 Fault를 임의로 낮추지 않음

01 CRITICAL_MASK = 3'b100

Device 2의 Timeout·Error·Critical을 모두 대상화

Count 대기 없이 Level 3

02 RESET_FAULT · W1P

활성 Fault가 하나라도 남아 있으면 명령 무시

입력 해제 후 Count · IRQ Pending Clear

03 IRQ_STATUS · W1C

Level·Device·Code 중 하나만 바뀌어도 Set

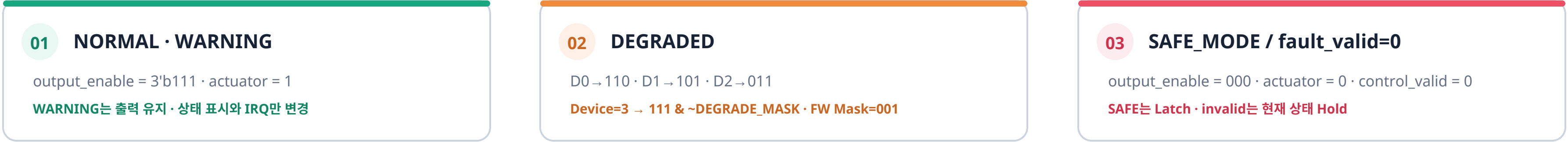
Pending 해제 전까지 Level IRQ 유지

안전 제어기 IP

Fault Manager의 판정값을 4-State FSM으로 해석하고, 상태별 Output Gate로 장치·구동기·제어 경로를 차단



상태별 안전 출력 상태 판정과 실제 출력 차단을 하드웨어에서 결합



대표 통합 동작 — 3채널 Fault 대응

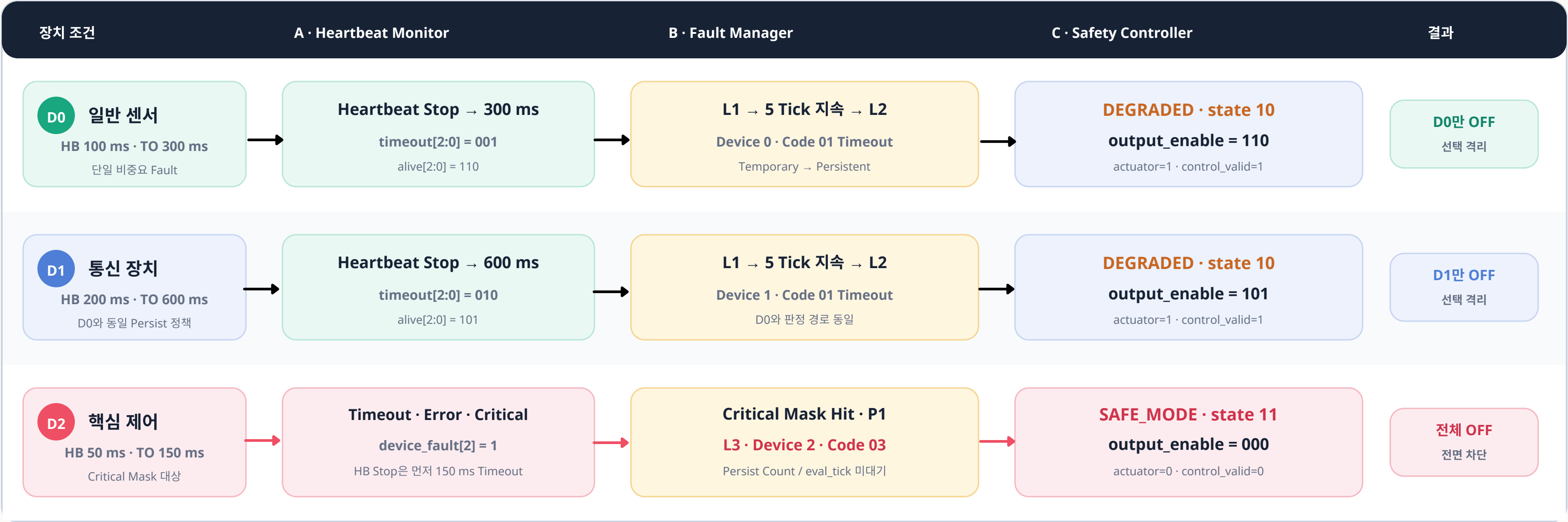
D0·D1은 지속 고장 장치만 선택 격리하고, D2는 Critical Mask로 전체 출력을 차단

eval_tick · 1 ms

PERSIST_LIMIT · 5

CRITICAL_MASK · 3'b100

실기 기본 설정 · output_enable[2:0] = D2 D1 D0



공통 경로 · D0 / D1

단일 일반 Fault → L1 WARNING → 5 Tick 지속 → L2 DEGRADED → 해당 장치만 차단

차이는 Timeout 설정과 Device 번호, 최종 output mask뿐이다.

분기 경로 · D2

Mask Hit → Persist 우회 → L3 SAFE_MODE → 000

“즉시”는 Fault가 B에 확정된 이후의 경로를 의미한다.

고장 정책 검증 매트릭스

RTL 기대값과 Fault Manager Self-checking Simulation 관측값을 같은 기준으로 비교

4,146
Behavioral Checks

0
Errors · ALL PASS

4,096
Reference Model 조합 전수 비교

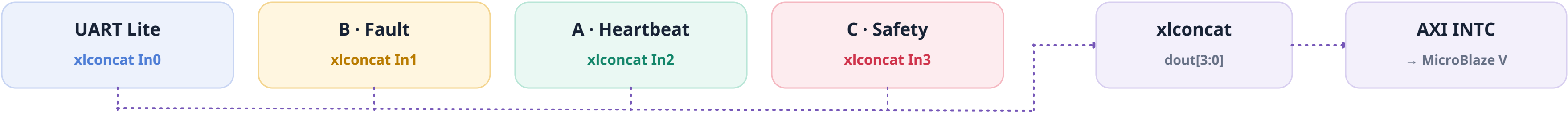
증거 범위
B 판정은 실행 결과 · C 상태/출력은 확정 RTL 정책 매핑

시나리오	판정 조건	B · Level / Device / Code	C 기대 상태	Output	결과
NORMAL	device_fault = 000	L0 · dev3(None) · 00	NORMAL	111	MATCH
D0 TRANSIENT	timeout[0]=1 · Count < Limit	L1 · dev0 · 01 Timeout	WARNING	111	MATCH
D0 PERSISTENT	timeout[0]=1 · Count ≥ Limit	L2 · dev0 · 01 Timeout	DEGRADED	110	MATCH
D2 CRITICAL	device_fault[2] & mask[2]	L3 · dev2 · 03 Critical	SAFE_MODE	000	MATCH
MULTI DEVICE	popcount(device_fault) ≥ 2	L3 · dev3(Multi) · 04	SAFE_MODE	000	MATCH

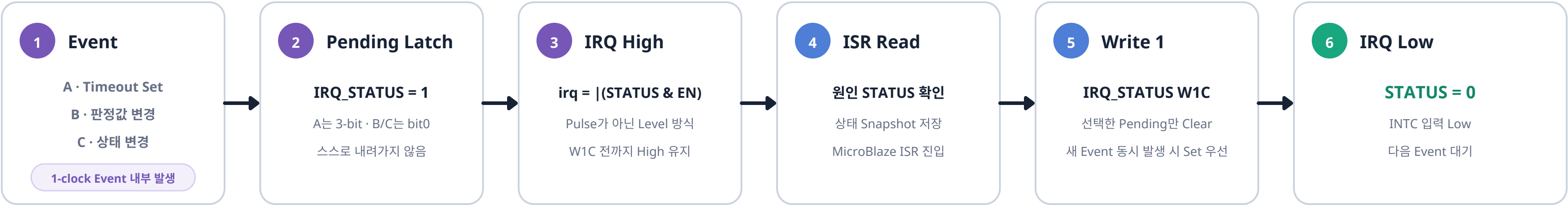
Level IRQ와 W1C 해제 검증

이벤트는 Pending에 저장하고, ISR이 원인을 읽은 뒤 선택 비트만 Write-1-to-Clear

IRQ SOURCE MAP · ACTUAL BLOCK DESIGN



LEVEL IRQ SEQUENCE



A · Heartbeat

W1C → Pending만 Clear

Timeout/Counter는 유지

CLEAR_ALL과 IRQ_STATUS 역할 분리

B · Fault Manager

W1C → Pending만 Clear

Level·Device·Code는 현재 입력 추적

Fault 원인을 제거하지 않음

C · Safety

W1C → Pending만 Clear

system_state는 그대로 유지

SAFE_MODE 해제와 무관

검증 상태

B AXI Main · 46/0 PASS

B IRQ Gating · 6/0 PASS

A/C·통합은 Assertion Source 확보

Raw 실행 로그는 저장소에 없음

GUI 화면 설명

GUI 는 모니터가 아니라 양방향 검증 단말입니다 · 관찰 · 설정 · 주입 · 제어 · 증빙을 하나의 UART 링크로 처리

NORMAL

화면 캡처 삽입

Event Log 가 함께 보이도록

DEGRADED

화면 캡처 삽입

Event Log 가 함께 보이도록

SAFE_MODE

화면 캡처 삽입

Event Log 가 함께 보이도록

Level 0 · oe = 0b111 · Actuator 1 · 3 채널 Alive

Level 2 · oe = 0b110 · D0 Timeout · Fault Count 증가

Level 3 · oe = 0b000 · Actuator 0 · Control Valid 0

① 양방향 검증 단말

관찰 · \$MISSION 500 ms 주기 · \$EVENT 전이 이벤트
설정 · SET 8종 (TIMEOUT×3 / MASK / LIMIT / COUNT / IRQ_EN)
주입 · INJECT Error · Critical · Heartbeat 중단
제어 · CMD 4종 (Manual Reset / Reset Fault / Clear IRQ / Clear HB)
증빙 · CSV 2종 (mission_log / mission_events)

② 값을 고치지 않는 앱

파서가 예외를 던지지 않는다 · 모르는 값은 UNKNOWN 표시
성공을 가정하지 않는다 · \$ACK 전에는 상태를 바꾸지 않음
POLICY WARNING · 명세 위반값도 보정 없이 기록만
색상만으로 구분하지 않는다 · [OK][!][!!][STOP]

③ 두 개의 표시 경로

큰 글씨 상태 ← \$MISSION · 500 ms 주기 · “지금 상태”
최근 전이 트레일 ← \$EVENT · 전이마다 즉시

WARNING 은 5 ms 라 큰 글씨엔 안 잡히지만
ISR Snapshot 경로로 Event Log 에는 반드시 남는다

안전 판단과 출력 차단은 전부 FPGA 내부 · 앱을 꺼도 SAFE_MODE 전환에 영향 없음 · Mock Simulator 모드로 보드 없이 정책 검증 가능

시연영상

검증 범위

계층마다 검증할 수 있는 것과 없는 것을 나누고, 못 하는 것은 아래 계층에 맡겼습니다

① RTL Testbench · sim/ 클럭 단위 · 전수 조합 · 경계조건 · TB 8 종 (core 4 · axi 3 · 통합 1)	Testbench tb_heartbeat_monitor_core tb_heartbeat_monitor_axi tb_fault_manager_core tb_fault_manager_axi (+ A11 / A12 / A13) tb_safety_controller_core tb_safety_controller_axi tb_eval_tick_generator tb_mission_soc_top AXI TB 3 종 공통 · A/W 도착 순서 변경 · WSTRB 부분 쓰기 · BREADY/RREADY backpressure · Protocol Monitor	검증 범위 · checks / fail Counter · Timeout · Alive · AUTO_RECOVER AXI R/W · W1P · W1C · Reset 기본값 정책 21 항목 + 전수 4,096 조합 — 4,146 / 0 AXI · IRQ Level · RESET_FAULT — 46 / 0 · 6 / 0 FSM 26 항목 · Recovery · SAFE Latch AXI R/W · W1P · W1C Reset 중 0 · 폭 1 clock — 5 / 0 BD net 배선을 그대로 옮긴 통합 TB
---	---	--

보드 + UART 로 검증할 수 없는 것 · 대신 어디서 검증했나

DEGRADED → WARNING 하강 persist_cnt 가 255 에서 포화하고 PERSIST_LIMIT 최대도 255 라 Fault 가 살아 있는 채로 Level 2 → 1 을 만들 UART 수단이 없다 → tb_safety_controller_core	클럭 단위 동시 다중 Fault UART 9600 bps · 관측 분해능이 ms 단위 두 INJECT 는 별개 줄이라 순차 처리된다 (실측 229 ms) → tb_fault_manager_core	20 ns 차단 지연 측정 같은 이유로 UART 로는 관측할 수 없다 동기화 + FM 1 clk + SC 1 clk 은 파형으로만 확인 → RTL 파형
--	--	---

트러블슈팅

세 번의 문제는 모두 “가정이 틀렸다” 에서 시작했습니다

01 짧은 상태가 로그에서 사라졌다

증상

STATE_CHANGE,WARNING 이 한 줄도 없음
FAULT_CHANGE 가 level 0 → 2 로 1 을 건너뛴

원인

메인 루프가 5 ms 폴링으로 직전 값과 비교하는 구조
Level 1 유지 = eval_tick 1 ms × PERSIST_LIMIT 5 = 5 ms
9600 bps 송신 블로킹까지 겹쳐 감지 지연 50 ms 이상
하드웨어는 0 → 1 전이에도 이미 IRQ 를 올리고 있었다

조치

ISR Snapshot Ring (깊이 16) 도입
IRQ 진입 순간의 상태값을 저장 → 메인 루프가 \$EVENT 생성
ISR 순서 : 원인 읽기 → W1C → Snapshot → 플래그
폴링은 백스톱으로만 유지

결과

기본값 PERSIST_LIMIT = 5 에서도 WARNING 정상 기록
Ring 초과 시 경고를 출력해 유실을 숨기지 않는다

02 경고 2,730 건의 원인은 XDC 한 줄

증상

TIMING-6 Critical Warning × 2
check_timing multiple_clock = 2,730

원인

BD 에 clk_wiz 가 있으면 Vivado 가 클럭 제약을 자동 생성
Digilent 마스터 XDC 의 create_clock -add 가 하나 더 추가
같은 포트에 클럭 2 개 → MMCM 출력도 _0 / _0_1 로 분리
도구가 만든 제약을 사람이 중복 정의한 것

조치

XDC 의 create_clock -add 줄 제거
primary clock 은 clk_wiz 가 만드는 것 하나만 유지
참고용 XDC 사본에도 같은 줄이 있어 경고 주석 추가
다시 넣으면 그대로 부활하므로 문서에 명시

결과

TIMING-6 2 → 0 · TIMING-56 2 → 0
multiple_clock 2,730 → 0

03 UART 송신이 수신과 Heartbeat 를 막았다

증상

GUI 버튼을 눌러도 명령이 씹힘
아무 조작 없이 Timeout 이 발생

원인

9600 bps 에서 \$MISSION 한 줄 = 약 73 ms 블로킹
그동안 UARTLite RX FIFO(16 byte)가 넘쳐 명령이 잘림
동시에 Heartbeat 생성이 멈춰 진짜 Timeout 이 판정됨
관측 행위가 관측 대상을 바꾼 경우

조치

RX Ring 256 byte · tx_putc() 가 글자마다 RX 를 퍼담음
모든 송신 함수가 진입 전 HBGEN_Pump() 호출
시간 판정 기준을 HW LAST_COUNTn 으로 변경
xil_printf 대신 자체 PROTO_Printf 구현

결과

명령 유실 0 · Timeout 오탐 0
CPU 가 멈춰 있어도 경과 시간이 정확

공통 교훈 · 하드웨어는 이미 알려주고 있었다 — 소프트웨어가 늦게 읽은 것이었다 · 도구가 만든 것을 사람이 또 만들지 않는다

합성 · Timing · 자원 사용량

100 MHz 목표를 만족했고, 저비용 FPGA 한 칩에 안전 경로와 제어 경로를 모두 넣었습니다

Vivado 2024.2 · Basys 3 xc7a35tcpg236-1 · Top mission_soc_wrapper · Synthesis Complete · write_bitstream Complete

___ ns

WNS · Worst Negative Slack

0.000 ns

TNS · Total Negative Slack

___ ns

WHS · Worst Hold Slack

0

Failed Routes · 100 MHz 목표 달성

RESOURCE UTILIZATION

항목	사용	전체	비율
Slice LUT	___	20,800	___
Slice Register	___	41,600	___
Block RAM Tile	32	50	64.0%
DSP	0	90	0.0%

report_utilization 의 Slice LUTs 행 기준 · LUT as Logic 과 다른 값이므로 혼용 금지

METHODOLOGY DRC

RESOLVED

TIMING-6 Critical Warning 2 → 0

multiple_clock 2,730 → 0

TIMING-56 2 → 0

REMAINING

TIMING-18 × 19 · 비동기 외부 I/O · 의도적 예외

LUTAR-1 × 2 · MicroBlaze V 디버그 유닛 내부 셀

전 항목 Related violations: <none> → 실제 타이밍 위반 경로 0 개

DSP 0

안전 판단에 곱셈이 없다 — 비교 · 카운터 · 비트마스크뿐

BRAM 64%

대부분 MicroBlaze V Local Memory 128 KB

여유 확보

Custom IP 3 개가 쓰는 자원은 작다 · 확장 여지 있음

WNS / WHS / Slice LUT / Slice Register 는 Vivado impl_1 재리포트 후 기입 — report_timing_summary 와 report_utilization 을 같은 실행에서 뽑을 것

감사합니다

QR 삽입